Kerbo-S Design Document, Version 2.0 (Sprint 1)

AI-Assisted Kerberos Security Auditor | CSCE 4907 Senior Capstone

GitHub URL: https://github.com/chancellorr/Cybersecurity-Capstone.git

Team: Chance Edwards, Ben Grider, David Pantoja, Jessica Yescas | Revision date: October 29, 2026

Purpose

Kerbo-S is built as a pipeline in which only the rule engine decides what a finding is, and every stage can be tested without the Kerberos lab. This document supersedes the architecture in Appendix B of the v1.0 proposal and implements Requirements Document v2.0. Implementation status: the MIT Kerberos lab VM exists; no Kerbo-S code is written yet, so everything below is the design to build in Sprint 2.

1. Design Drivers

- **Determinism** (FR-AUD-02, NFR-AUD-01): identical input data and thresholds must give identical findings.
- **Safety** (NFR-SEC-01, FR-AI-02): collection is read-only, and the AI is isolated from every decision.
- **Testability** (FR-COL-08, NFR-MNT-01): all logic runs from stored fixtures, so no one needs the VM to develop or test.
- **Deliverability:** four developers and limited time favor small, well-known tools over clever ones.

2. Design Evolution from v1.0

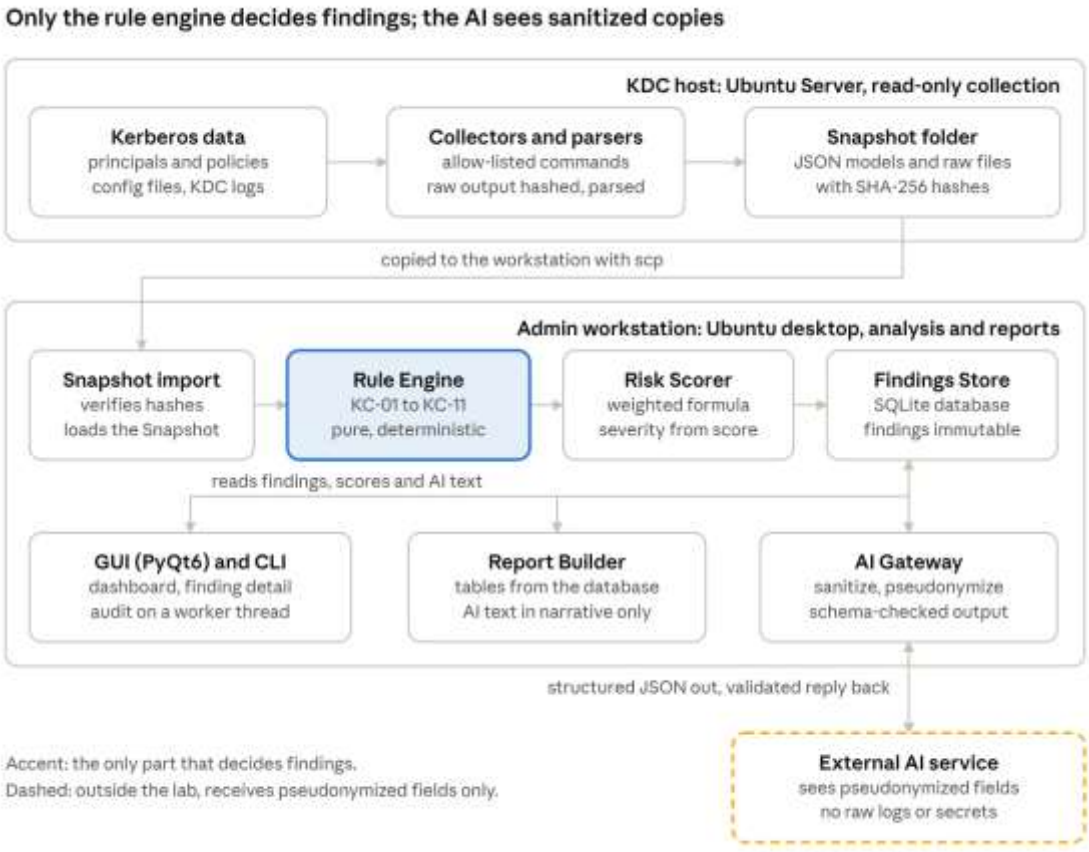
v1.0 described a five-layer stack (interface, audit management, collection, analysis engine, AI engine) with SQLite at the bottom. The layers were right in spirit, but the design left stage boundaries, data formats and the AI boundary undefined. The table lists what changed and what each change costs.

ID	v1.0 design	v2.0 design	Reason	Trade-off
D-01	Layers connected by direct calls	Pipeline with typed hand-offs (Snapshot, Finding, AiOutput) and a persisted result at each stage	Each stage becomes testable alone and reproducible	More up-front plumbing and data classes
D-02	Collection read the live KDC during every audit	Collectors write hashed raw artifacts and a normalized Snapshot; the rule engine reads only the Snapshot	Develop and test without the VM; reproducible audits; tamper evidence	Two code paths (live and import) and extra disk use
D-03	AI layer after scoring, no defined boundary	AI Gateway with sanitizer, pseudonymizer, versioned prompts, provider interface (OpenAI, mock) and schema validation	Addresses prompt injection and data exposure; AI can be tested offline	Extra component; free-form AI answers are not allowed

ID	v1.0 design	v2.0 design	Reason	Trade-off
D-04	Fixed thresholds and severities in rule text	Thresholds and risk-factor baselines in validated config; severity derived from score	Fixes the KC-02 contradiction; allows tuning in the lab	A bad config could skew results, so it is validated on load
D-05	Single GUI process, Linux and Windows	Headless core library with GUI and CLI front ends; audit runs on a worker thread; Linux only	Responsive UI; the KDC VM has no desktop; same code under test	Thread handling adds complexity
D-06	Report generation inside the AI layer	Separate Report Builder: tables come from the database, AI supplies text only; digest embedded	A report must work with the AI down and be verifiable	Two sources feed each report
D-07	Generic SQLite store	Defined schema; findings immutable through database triggers	Meets FR-DB-02 and TM-05	Triggers must be tested; a local admin can still replace the file

3. System Architecture

The core library contains no GUI code, so the PyQt6 application, the command-line tool and the test suite all run the same logic. The diagram shows the components, the data stores and the three trust zones.



architecture · 3 trust zones

Collection and parsing run on the KDC host; everything after the snapshot copy runs on the workstation, and only sanitized, pseudonymized fields reach the external AI service.

Component	Responsibility	Key interface	Requirements
GUI (PyQt6)	Run audits, dashboard, finding detail, reports, settings	Calls AuditService on a worker thread	FR-UI-01 to 04, NFR-PERF-02
CLI	Headless collect, export snapshot, audit; used on the KDC and in tests	Calls AuditService	FR-UI-05, FR-COL-08
AuditService	Orchestrates stages, records errors, owns the audit record	collect, audit, analyze, report	FR-AUD-01, NFR-REL-01
Collectors	Run allow-listed commands or read files; hash and store raw artifacts	collect() returns RawArtifact	FR-COL-01 to 07, 09
Parsers	Turn raw text into typed models	parse(artifact) returns models	FR-COL-01 to 05, NFR-SEC-04
Snapshot I/O	Export and import snapshot folders;	export(), import_()	FR-COL-08, TV-

Component	Responsibility	Key interface	Requirements
	verify hashes		14
Rule Engine	Eleven independent rule classes over a Snapshot	evaluate(snapshot, config) returns findings	FR-AUD-01 to 07
Risk Scorer	Applies factor baselines and the weighted formula; derives severity	score(finding)	FR-RSK-01 to 04
Findings Store	SQLite repository; findings immutable	save and query methods	FR-DB-01, 02
AI Gateway	Sanitize, build prompt, call provider, validate, retry, fall back	Provider.complete(prompt)	FR-AI-01 to 09
Report Builder	Technical and executive reports, PDF output, digest	build(kind) returns PDF	FR-RPT-01 to 05
Config Loader	Loads and validates thresholds, factor baselines and settings	load()	FR-AUD-03, FR-RSK-03

4. Deployment View

The lab KDC runs Ubuntu Server with no desktop, so the GUI cannot run there. The design therefore supports two cooperating modes, and the KDC host never needs internet access or an AI key.

Mode	Where	Steps	Purpose
Headless collection	KDC VM, run with sudo	Run the CLI collect command with the export option; copy the snapshot folder to the workstation with scp	Gather live data read-only
Analysis and reporting	Ubuntu desktop admin VM	Import the snapshot (hashes verified); audit; AI analysis; review; PDF export	Day-to-day use and demos

Trade-off: the copy step is manual, but the same snapshot also serves as the test fixture and the evidence trail.

5. Technology Choices

Concern	Choice	Why	Rejected alternative
Language	Python 3.10 or later	Default on Ubuntu LTS; team familiarity	None
GUI	PyQt6	In the approved proposal; worker threads and PDF	Tkinter (weak tables); web app (out of scope)

Concern	Choice	Why	Rejected alternative
		writer included	
Storage	SQLite via `sqlite3`	No administration; supports triggers; one file	JSON files (no queries); PostgreSQL (overkill)
Configuration	YAML, validated into dataclasses	Human-editable	Hard-coded constants (breaks FR-AUD-03)
Snapshot format	JSON plus raw text files	Diffable, safe to load	Pickle (unsafe to load)
AI	OpenAI Python SDK behind a Provider interface	In the approved proposal; JSON-schema output	Local Llama as primary (hardware and quality unknown; kept as R2)
PDF	QTextDocument rendered with QPdfWriter	No extra dependency	ReportLab (more layout control; fallback if needed)
Testing	pytest with fixtures captured from the lab	Simple, parameterized tests	unittest
Source control	Git and GitHub with pull-request review	Required; gives commit history	None

The OpenAI model is a config value (`ai.model`). It is chosen in Sprint 2 by running the Requirements Document Section 7.2 evaluation on two candidates and keeping the cheapest that reaches 90% accuracy with valid output; the result will be recorded here (Open Question 1).

6. Data Design

6.1 Normalized models

- **Principal:** name, kind (user, service, admin, internal), last password change, last successful authentication, maximum ticket life, key encryption types, attributes, policy name.
- **Policy:** name, minimum length, minimum and maximum life, history, failure-count limit.
- **ConfigSetting:** file, section, key, value.
- **AdminEntry:** principal, permissions, justified (true or false).
- **AuthEvent:** timestamp, principal, client address, request type, result.
- **Snapshot:** all of the above plus collection timestamp and artifact hashes.

6.2 SQLite schema

Table	Main columns	Notes
audits	id, started_at, mode, tool_version, ruleset_version, config_hash	One row per audit
artifacts	id, audit_id, source, command_or_path,	Raw evidence

Table	Main columns	Notes
	collected_at, sha256, stored_path	
findings	id, audit_id, rule_id, object_type, object_id, observed, threshold, impact, exploitability, exposure, compliance, score, severity, artifact_id	Immutable: triggers block UPDATE and DELETE
reviews	finding_id, status, note, reviewed_at	The only mutable finding data
ai_outputs	id, audit_id, finding_id, kind, payload_json, model, prompt_version, status, created_at	Provenance for every AI text
reports	id, audit_id, kind, path, findings_digest, created_at	Digest supports the verify function
collection_errors	id, audit_id, source, message, occurred_at	Feeds the error list in reports

7. Component Design

7.1 Collectors and parsers

- Commands come from a fixed allow-list and run as argument lists with no shell; principal and policy names are checked against a strict pattern first.
- Each command has a 30-second timeout and a 10 MB output cap.
- The raw output is written to the artifact folder and hashed before parsing.
- Parsers use strict patterns; a line that does not match is logged as an error, not fatal.
- Parser tests run against real output captured from the lab VM and saved as fixtures.

7.2 Rule engine

Each rule is a small class with no input or output of its own, so it is easy to test and deterministic. Ages are measured against the snapshot's collection time, never the current clock, so a stored snapshot always gives the same result.

```
class KC01PasswordAge(Rule):
    id = 'KC-01'
    def evaluate(self, snap, cfg):
        limit = cfg.threshold('KC-01', 'max_age_days')  # default 90
        for p in snap.principals:
            age = p.password_age_days(snap.collected_at)
            if p.kind == 'user' and age > limit:
                yield Finding(self.id, 'principal', p.name, observed=age,
threshold=limit)
```

The engine runs rules in ID order and sorts findings by rule and object, so output order is stable.

7.3 Risk scorer

The scorer reads each rule's baseline Impact, Exploitability, Exposure and Compliance values from config, applies Impact x 0.40 + Exploitability x 0.30 + Exposure x 0.20 + Compliance x 0.10, and maps the score to Low, Medium, High or Critical. The factor values are stored with each finding.

7.4 AI gateway

1. **Select:** only validated findings with their stored fields are eligible.
2. **Sanitize:** keep an allow-list of fields (rule, severity, score, observed value, threshold, object type); drop free text such as descriptions and log messages.
3. **Pseudonymize:** replace principal and host names with tokens such as P001; the mapping stays local and is reversed only when displaying results.
4. **Build prompt:** a versioned template puts instructions in the system role and the finding data as a JSON block in the user role, with an instruction to treat it as data.
5. **Call:** the provider interface (OpenAI or mock) runs with temperature of 0.2 or lower and a JSON schema (explanation, risk context, remediation steps, related groups, remediation order).
6. **Validate:** schema failures are retried once, then stored as unavailable; any field that tries to set a score or severity is rejected.
7. **Store and label:** the output is saved with model, prompt version and timestamp, shown as AI-generated and advisory.

A response cache keyed by the hash of the sanitized input and prompt version saves cost and keeps repeat text stable. It is switched off during the consistency evaluation. If the provider is unreachable, the audit, scoring and report still complete without AI sections.

7.5 Report builder

Tables of findings, scores and errors are generated from the database. AI text fills only the narrative sections. The builder renders HTML into a QTextDocument, writes the PDF with QPdfWriter, embeds the SHA-256 digest of the finding set, and records the digest in the reports table.

7.6 User interface

Screen	Contents	Requirements
Run Audit	Choose live or snapshot, start, progress bar, error list	FR-UI-01
Findings Dashboard	Table sorted by score, severity and category filters, counts by severity	FR-UI-02
Finding Detail	Observed value, threshold, evidence reference, score factors, AI analysis, review status and note	FR-UI-03, 04
Reports	Generate technical or executive report, export PDF, verify digest	FR-RPT-01 to 05

Screen	Contents	Requirements
Settings	Paths, thresholds (read-only view in MVP), AI provider and model	FR-AUD-03, FR-AI-04

7.7 Concurrency and errors

An audit runs on a QThread worker that emits progress and error signals, so the interface never blocks. Every stage is wrapped so a failure is stored in collection_errors and the next stage still runs (NFR-REL-01).

8. Key Flows

Audit run

1. The user chooses live collection or a snapshot and starts the audit.
2. Collectors capture raw artifacts and hashes, or Snapshot I/O verifies the imported hashes.
3. Parsers produce the Snapshot.
4. The rule engine evaluates KC-01 to KC-11.
5. The risk scorer scores each finding and the Findings Store saves it.
6. The dashboard displays results.

AI analysis and reporting

1. The user selects Generate AI Analysis.
2. The gateway sanitizes, pseudonymizes, calls the provider and validates the result.
3. The user reviews findings and AI text and marks reviews.
4. The user generates a report; the builder renders the PDF and stores its digest.

9. Security Design

Control	Implemented in	Threat
Allow-listed, read-only commands; no shell	Collectors	TM-01, TM-07
Raw artifact hashing; hash check on import	Collectors, Snapshot I/O	TM-01, TM-05
Field allow-list, pseudonymization, fixed prompts, schema validation	AI Gateway	TM-02, TM-03
API key from an environment variable; never written to config or the repository	Config Loader	TM-04
Owner-only permissions on database, snapshots and reports	Findings Store, Report Builder	TM-06
Immutable findings and report digest with verify function	Findings Store, Report Builder	TM-05

Control	Implemented in	Threat
KDC host never calls the AI service	Deployment view	TM-03

10. Design Decisions and Trade-offs

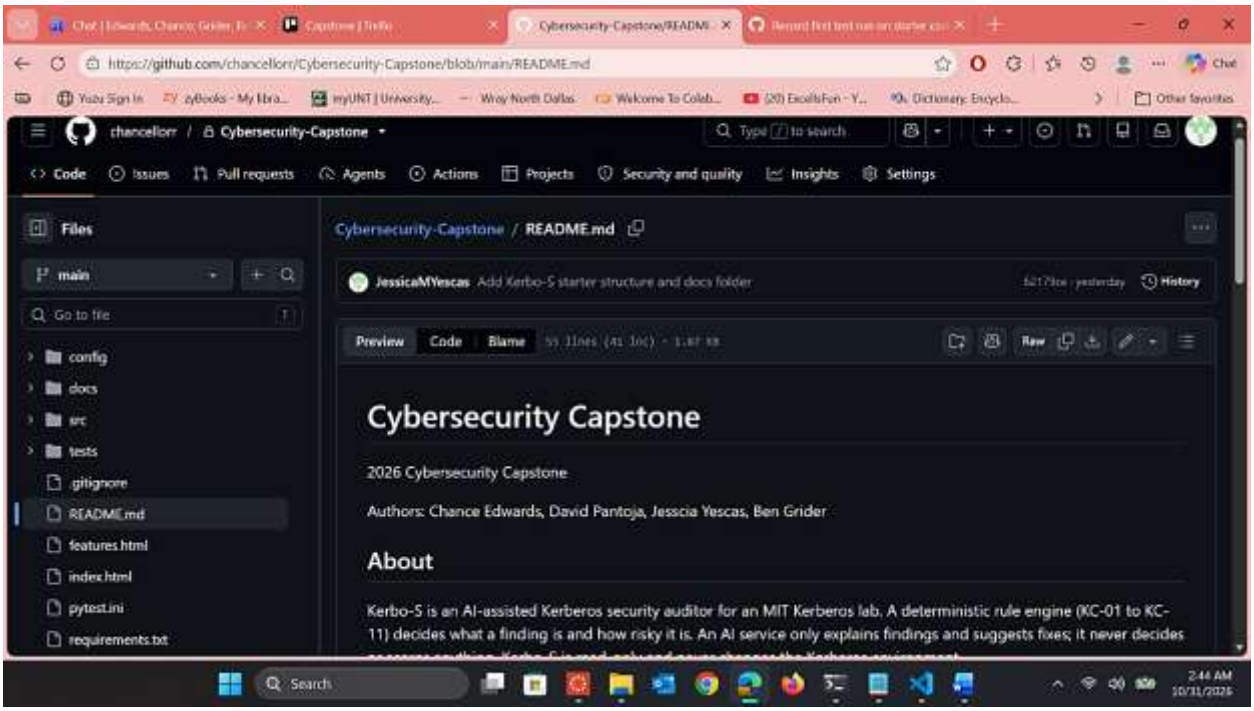
Decision	Advantages	Disadvantages and risks	Mitigation
Deterministic engine separate from AI	Reproducible, auditable findings; AI failures cannot change results	AI cannot discover issues the rules miss	Add rules as new weaknesses are found; AI correlation adds context
Snapshot boundary between collection and analysis	Test without the VM; evidence trail; split deployment	Manual copy step; stale snapshots	Show collection time on every screen and report
Parse `kadmin.local` text output	Works on any MIT Kerberos install; no extra libraries	Output format may vary between versions	Fixtures from the lab; strict patterns; errors logged
Rules as Python classes with YAML thresholds	Easy unit tests; thresholds tunable	New rule needs code; not a declarative rule language	Small base class and template for new rules
OpenAI as primary AI behind an interface	Best expected quality for the effort; swap-in local model later	Data leaves the lab; cost; internet dependency	Sanitize and pseudonymize; mock mode; response cache
Pseudonymize before AI	Reduces exposure of real names	Explanations are less specific; mapping must be protected	Keep mapping local; re-insert names when displaying
SQLite with triggers	Simple; no server; tamper-resistant at the row level	A local admin can replace the file	Digest verification; owner-only permissions
Linux only	Matches the lab; fewer test cases	Narrower audience than v1.0	Windows clients still audited through KDC logs; Windows host under Future Enhancements
Qt-based PDF output	No extra dependency	Less layout control	Switch to ReportLab if layout is inadequate

11. Requirements Traceability

Requirement group	Design elements
FR-COL (collection)	Collectors, Parsers, Snapshot I/O, artifacts table
FR-AUD (audit engine)	Rule Engine, Config Loader, findings table
FR-RSK (risk scoring)	Risk Scorer, config baselines
FR-AI (AI analysis)	AI Gateway, ai_outputs table
FR-RPT (reporting)	Report Builder, reports table
FR-UI and FR-DB	GUI screens, CLI, Findings Store
NFR-SEC and TM-01 to 07	Section 9 controls
NFR-PERF, NFR-MNT	Worker thread, independent rule classes, fixtures

12. Repository Layout

```
kerbo-s/
  README.md
  requirements.txt
  config/            rules.yaml, app.yaml, admin_justifications.example.yaml
  src/kerbo_s/
    collectors/      principals.py, policies.py, config_files.py, logs.py
    parsers/         kadmin.py, kerberos_conf.py, kdc_log.py
    snapshot/        export_import.py
    rules/           base.py, kc01.py ... kc11.py
    scoring/         scorer.py
    store/           schema.sql, repository.py
    ai/              gateway.py, sanitizer.py, prompts/, providers/
    reports/         builder.py
    ui/              main_window.py, dashboard.py
    cli.py
  tests/
    fixtures/        snapshots and expected_findings manifests
    test_rules/  test_parsers/  test_scoring/  test_ai/
  docs/              requirements, design, test plan, change document
```



13. Risks and Open Questions

1. Final OpenAI model and its cost per audit (to be set after the Sprint 2 pilot).
2. Exact `kadmin.local` field labels and KDC log message formats on the lab's Kerberos version; confirm from captured fixtures.
3. Whether last-authentication data supports KC-04 as designed, or the KDC log must be used as well.
4. Whether the GUI can be installed on the lab's desktop VM without extra system packages.
5. KC-11 default threshold, to be set after observing normal lab traffic.